

9. Java

```
// ClaseA.java
public class ServicioA {
    private ServicioB servicioB;

    public ServicioA(ServicioB b) { this.servicioB = b; }

    public String procesarPaso1(String data) {
        // 20 líneas de lógica compleja
        if (data == null || data.length() < 5) {
            throw new IllegalArgumentException("Data inválida");
        }
        String resultado = data.toUpperCase();
        for (int i = 0; i < 3; i++) {
            resultado += "-procesado";
        }
        // Llama a B (flecha A→B)
        return servicioB.procesarPaso2(resultado);
    }

    public void finalizar() {
        // Llamado por C (cierre del ciclo C→A)
        System.out.println("ServicioA finalizado");
    }
}

// ClaseB.java
public class ServicioB {
    private ServicioC servicioC;

    public ServicioB(ServicioC c) { this.servicioC = c; }

    public String procesarPaso2(String data) {
        // 18 líneas idénticas
        if (data.length() < 10) throw new IllegalArgumentException();
        String resultado = data.replace("-", "_");
        resultado += "_B";
        // B→C
    }
}
```

```

        return servicioC.procesarPaso3(resultado);
    }
}

// ClaseC.java
public class ServicioC {
    private ServicioA servicioA;

    public ServicioC(ServicioA a) { this.servicioA = a; }

    public String procesarPaso3(String data) {
        // 20 líneas
        String resultado = data + "_C_COMPLETO";
        // ¡CICLO CERRADO! C→A
        servicioA.finalizar();
        return resultado;
    }
}

// MainProblematico.java
public class Main {
    public static void main(String[] args) {

    }
}

```

10. Python

```
# Controller (capa 1 middle man)
class UsuarioController:
    def __init__(self):
        self._service = UsuarioService()

    def obtenerUsuario(self, id):
        # Solo delega (línea 6)
        return self._service.obtenerUsuario(id)

    def crearUsuario(self, data):
        return self._service.crearUsuario(data)

    def actualizarUsuario(self, id, data):
        return self._service.actualizarUsuario(id, data)

    def eliminarUsuario(self, id):
        return self._service.eliminarUsuario(id)

# Service (capa 2 middle man - 22 líneas)
class UsuarioService:
    def __init__(self):
        self._repo = UsuarioRepository()

    def obtenerUsuario(self, id):
        # IDENTICA delegación
        return self._repo.findById(id)

    def crearUsuario(self, data):
        usuario = Usuario(data['nombre'], data['email'])
        return self._repo.save(usuario)

    def actualizarUsuario(self, id, data):
        usuario = self._repo.findById(id)
        usuario.nombre = data['nombre']
        return self._repo.update(usuario)

    def eliminarUsuario(self, id):
        return self._repo.delete(id)
```

```
# Repository (capa 3 - REAL lógica)
class UsuarioRepository:
    def findById(self, id): return f"Usuario {id}"
    def save(self, u): return u
    def update(self, u): return u
    def delete(self, id): return True
```

11. C#

```
public class RegistroUsuario {
    public bool Registrar(string nombre, string apellido,
                          string email, string dni, string telefono,
                          string fechaNacimiento, string direccion) {

        if (string.IsNullOrEmpty(email) || !email.Contains("@") ||
            !email.Contains(".") || email.Length > 100) {
            throw new ArgumentException("Email inválido");
        }

        if (string.IsNullOrEmpty(dni) || dni.Length != 10 || !dni.All(char.IsDigit)) {
            throw new ArgumentException("DNI inválido");
        }

        if (string.IsNullOrEmpty(telefono) || telefono.Length != 10 ||
            !telefono.All(char.IsDigit)) {
            throw new ArgumentException("Teléfono inválido");
        }

        if (!DateTime.TryParse(fechaNacimiento, out DateTime fecha)) {
            throw new ArgumentException("Fecha inválida");
        }

        return GuardarUsuario(nombre, apellido, email, dni, telefono, fecha, direccion);
    }

    private bool GuardarUsuario(string n, string a, string e, string d,
                                string t, DateTime f, string dir) {
        return true;
    }
}
```

12. JavaScript

```
class ConfiguracionSistema {
    // HARD-CODED EN TODAS PARTES
    static DB_HOST = '192.168.1.100';
    static DB_PORT = 5432;
    static DB_USER = 'admin';
    static DB_PASS = '123456';
    static API_KEY = 'sk-abc123def456';
    static MAX_CONEXIONES = 100;
    static TIMEOUT = 30000;

    static conectarDB() {
        return
        `jdbc:postgresql://${ConfiguracionSistema.DB_HOST}:${ConfiguracionSistema.DB_PORT}/sistema`;
    }
}

class ServicioEmail {
    enviar(email, mensaje) {
        const smtpHost = 'smtp.gmail.com';
        const smtpPort = 587;
        const user = 'noreply@empresa.com';
        const pass = 'GMAIL_APP_PASSWORD_123';
    }
}

class ApiExterna {
    async obtenerDatos() {
        const url =
        `https://api.externa.com/v1/data?key=${ConfiguracionSistema.API_KEY}`;
        // 20 líneas de fetch con timeouts hardcodedos
    }
}
```

13. Java

```
public class Calculadora {  
    /**  
     * Esta función suma dos números enteros.  
     * Recibe a y b como parámetros de entrada.  
     * Retorna la suma de ambos números.  
     * IMPORTANTE: No maneja números negativos.  
     */  
    public int sumar(int a, int b) { // OBVIO  
        return a + b; // Suma los dos parámetros  
    }  
  
    /**  
     * Esta función muy compleja hace lo siguiente:  
     * 1. Toma un número  
     * 2. Lo multiplica por sí mismo  
     * 3. Retorna el cuadrado  
     * ATENCIÓN: Puede dar overflow con números grandes  
     */  
    public long cuadrado(long numero) {  
        // Multiplica el número por sí mismo  
        return numero * numero; // ¡OBVIO!  
    }  
  
    /**  
     * VALIDACIÓN CRÍTICA: Verifica si email contiene @ y .  
     * Esta validación es MUY importante para seguridad.  
     * Sin esta validación el sistema colapsa.  
     * REVISADO POR: Juan Pérez 15/03/2024  
     */  
    public boolean esEmailValido(String email) {  
        // Verifica presencia de @ y punto  
        return email != null && email.contains("@") && email.contains(".");  
    }  
}
```